



Project File Guide & Library Reference

Phase 1 — Foundation & Data Core · April 2026

A plain-English walkthrough of every file, folder, and library in the backend

14%	Phase 1	■ Ahead	10 files
Overall done	Complete	13 days early	Core modules

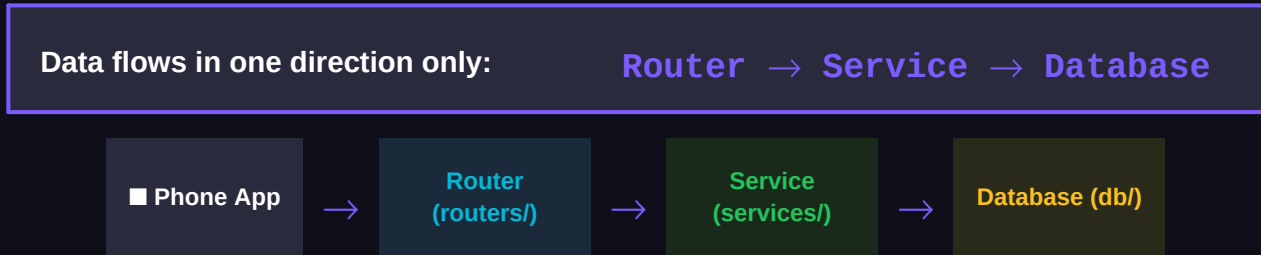
01. How the Project is Organised

The backend follows a clean layered architecture. Each folder has one job and one job only. This makes the code easy to find, easy to edit, and easy for Claude Code to extend without breaking things.

orbi-backend/	Root of the entire backend project
main.py	Entry point — starts the server
requirements.txt	List of all Python libraries needed
.env	Secret keys — never shared or committed
.env.example	Template showing which keys are needed
app/	All application code lives here
routers/	HTTP endpoints — what the app exposes
tasks.py	Task bubble endpoints
finance.py	Finance entry endpoints
users.py	User profile endpoints
services/	Business logic — the brain of the app
scoring.py	Pressure score calculator
ai_router.py	Routes AI calls to Groq or Claude
finance_categorizer.py	Rule-based merchant categorization
models/	Data shapes — what objects look like
task.py	TaskBubble, Cluster shapes
finance.py	FinanceEntry, Budget shapes
user.py	UserProfile, Preferences shapes
db/	Database — reads and writes to Supabase
client.py	Supabase connection setup
tasks.py	Task database queries
finance.py	Finance database queries
users.py	User database queries
agents/	AI agents — each has a specific job
coordinator.py	Routes user intent to right agent
task_parser.py	Converts speech to TaskBubble
finance_agent.py	Categorizes and analyses finances
prompts/	AI instructions stored as text files
task_parser_v1.md	Instructions for task parsing AI
finance_agent_v1.md	Instructions for finance AI
debrief_agent_v1.md	Instructions for debrief AI
tests/	Automated tests
test_scoring.py	Tests for pressure score logic

02. The Golden Rule of This Architecture

Before diving into individual files, understand this one rule that governs every decision in the codebase:



What this means in practice:

- The **router** receives the HTTP request and calls a service function. That's all it does.
- The **service** contains the logic — it does the thinking, calls AI if needed, and calls the database.
- The **database module** runs the actual query and returns raw data.
- This way, if you change the database, you only touch db/. If you change business logic, you only touch services/. Nothing breaks unexpectedly.

03. Every File Explained

`main.py`

The front door of the entire application

This is the first file Python runs when you start the server. It creates the FastAPI app, registers all the routers (tasks, finance, users), sets up CORS so the phone app can talk to the backend, and defines what happens on startup and shutdown.

- CORS middleware: allows the React Native app to make requests to this server
- Lifespan: runs setup code when server starts (e.g. test database connection)
- /health endpoint: lets you quickly check the server is alive — you already tested this
- Without this file, there is no server

`requirements.txt`

The shopping list of Python libraries

Python doesn't come with everything built in. This file lists every external library the project needs. When you run 'pip install -r requirements.txt', Python downloads and installs all of them at once.

- Every developer or server that runs Orbi starts by installing from this file
- Version numbers (e.g. fastapi==0.115.0) ensure everyone uses the exact same version
- If a library is missing from here, the app will crash when it tries to use it

`.env` / `.env.example`

Secret keys and configuration — kept private

`.env` holds your real secret keys (Supabase, Groq, Anthropic). It is never shared or committed to git. `.env.example` is a safe template that shows which keys are needed without revealing the actual values.

- SUPABASE_URL: where your database lives
- GROQ_API_KEY: unlocks the Llama AI model for free/pro users
- ANTHROPIC_API_KEY: unlocks Claude for premium users only
- ENVIRONMENT: tells the app if it's running in development or production
- Rule: if a key is not in `.env`, that feature will not work

`app/routers/tasks.py`

Defines the task endpoints the phone app can call

A router is like a receptionist. When the phone app sends a request to `/api/v1/tasks`, this file receives it, checks the user is logged in, and passes the work to the tasks service. It never does the actual thinking itself.

- GET `/api/v1/tasks` — returns all your bubbles ordered by pressure score
- POST `/api/v1/tasks` — creates a new bubble from a request
- PATCH `/api/v1/tasks/{id}` — updates an existing bubble
- DELETE `/api/v1/tasks/{id}` — archives a bubble (soft delete, data is kept)
- Every endpoint checks the Authorization header before doing anything

`app/routers/finance.py`

Defines the finance endpoints the phone app can call

Same pattern as `tasks.py` but for money. The phone app calls these endpoints to log expenses, check spending summaries, and manage budgets.

- POST `/api/v1/finance/entries` — log 'I spent £45 at Tesco'
- GET `/api/v1/finance/entries` — list all entries with optional filters
- GET `/api/v1/finance/summary` — monthly totals grouped by category
- POST `/api/v1/finance/budgets` — set a budget limit for a category
- Automatically categorizes merchant names before saving

`app/routers/users.py`

Defines user profile and preference endpoints

Handles getting and updating the user's profile, subscription tier, and preferences like quiet hours and how proactive the assistant should be.

- GET `/api/v1/users/me` — returns the current user's profile
- PATCH `/api/v1/users/me` — update name, preferences, notification settings
- The subscription tier stored here controls which AI model gets used

`app/services/scoring.py`

Calculates how urgent and important each bubble is

This is pure maths — no AI involved. It takes a `TaskBubble` and returns a number between 0 and 10 called the pressure score. The higher the score, the bigger and brighter the bubble appears in the universe.

- `deadline_weight`: score increases as due date gets closer
- `importance_weight`: based on how important you marked the task
- `attention_decay`: boosts tasks you have been ignoring for days
- `cluster_pressure_modifier`: if the Finance cluster is busy, all finance bubbles rise
- Fully deterministic — same inputs always produce same output, easy to test and debug

[app/services/ai_router.py](#)

The single gateway for all AI calls in the backend

Instead of calling Groq or Claude directly from random places in the code, every AI call goes through this one file. It checks the user's subscription tier and routes to the correct AI provider automatically.

- Free or Pro user → calls Groq API → uses Llama 3.1 8B → costs almost nothing
- Premium user → calls Anthropic API → uses Claude Sonnet 4.6 → costs more but more powerful
- If either provider fails, returns a safe fallback message instead of crashing
- Logs every call: which provider, which model, how many tokens, how long it took
- `load_prompt()` reads the correct .md file from `app/prompts/` and returns it as text

[app/services/finance_categorizer.py](#)

Sorts expenses into categories using rules before touching AI

When you log 'I spent £45 at Tesco', this service checks a list of known merchants and immediately assigns the category 'groceries' without calling any AI. Only if the merchant is completely unknown does it escalate to the AI.

- Tesco, Sainsbury's, Lidl, Aldi → Groceries
- Netflix, Spotify, Apple → Subscriptions
- Uber, TfL, Shell → Transport
- This saves AI costs on the vast majority of everyday expenses
- Unknown merchants get category 'uncategorized' and can be AI-classified later

[app/models/task.py](#)

Defines the exact shape of a TaskBubble

A model is a blueprint. It tells the app exactly what fields a TaskBubble must have, what type each field is, and which fields are optional. If the app receives data that doesn't match this shape, it rejects it automatically with a clear error.

- TaskBubble: the full object stored in the database
- TaskBubbleCreate: what the phone sends when creating a new bubble (no id yet)
- TaskBubbleUpdate: what the phone sends to edit a bubble (all fields optional)
- `owner_id` and `visibility` are here from day one, ready for the sharing feature later
- Pydantic v2 validates all data automatically — wrong type = instant error

`app/models/finance.py`

Defines the shape of finance objects

Same idea as `task.py` but for money. Defines exactly what a `FinanceEntry`, `FinanceBudget`, and `FinanceInsight` look like.

- `FinanceEntry`: one transaction — amount, merchant, category, date
- `FinanceEntryCreate`: the shape when logging a new expense
- `FinanceBudget`: a monthly spending limit for one category
- `FinanceInsight`: an AI-generated observation like 'food spending up 40%'
- `EntryType` enum ensures type is always 'income' or 'expense', never a typo

`app/models/user.py`

Defines the shape of user objects

Describes what a `UserProfile` and `UserPreference` look like. The `subscription_tier` field here is the most important one — it controls which AI the user gets.

- `SubscriptionTier` enum: free, pro, premium — no other values allowed
- `UserProfile`: id, email, name, tier, timestamps
- `UserPreference`: quiet hours, proactivity level, reminder channel
- These preferences will control when and how the assistant speaks to you

`app/db/client.py`

Opens the connection to the Supabase database

Creates one Supabase client that every other db file imports and uses. This means the connection is opened once, not hundreds of times.

- Reads `SUPABASE_URL` and `SUPABASE_SERVICE_KEY` from `.env`
- Exports a single 'supabase' object imported everywhere else in `db/`
- `test_connection()` lets `main.py` check on startup that the database is reachable
- If this file breaks, no data can be read or written anywhere

`app/db/tasks.py` / `finance.py` / `users.py`

All database queries for each domain

These files contain the actual SQL-style queries that read and write to Supabase. Service files call functions in here rather than querying the database directly.

- `get_tasks_by_user(user_id)` — fetches all bubbles for one user
- `create_task(task_data)` — inserts a new bubble into the database
- `update_task(task_id, updates)` — changes fields on an existing bubble
- Keeping queries here means if you change database provider you only edit these files

[app/agents/coordinator.py](#)

The traffic controller for AI conversations

When a user says something, the coordinator reads the intent and decides which specialist agent should handle it. It owns the conversation and keeps it coherent.

- 'Add dentist appointment Friday' → routes to task_parser
- 'I spent £30 at McDonald's' → routes to finance_agent
- 'How did the meeting go?' → routes to debrief_agent
- 'What did I spend on food this month?' → routes to finance_agent with query mode
- The user experiences one seamless assistant even though many agents work behind it

[app/agents/task_parser.py](#)

Converts natural language into a structured TaskBubble

When you say 'remind me to call the insurance company before Thursday', this agent uses the Groq/Claude AI to extract: title, due date, domain, importance. It then returns a TaskBubbleCreate object ready to be saved.

- Reads its instructions from app/prompts/task_parser_v1.md
- Calls ai_router.py — never calls Groq or Claude directly
- If confidence is low, flags the result for user confirmation
- Stores the original speech alongside the structured result for memory

[app/agents/finance_agent.py](#)

Handles all financial intelligence

This agent does three jobs: categorize new entries, generate spending insights, and detect anomalies. It runs automatically when new entries arrive and on a daily schedule to surface trends.

- Categorize: tries rule-based first, calls AI only for unknown merchants
- Insights: 'You spent 40% more on dining this month vs last month'
- Anomaly: 'This £800 charge at Amazon is much higher than your usual spend'
- Budget alerts: 'Food budget is 80% used with 12 days left in the month'
- Reads instructions from app/prompts/finance_agent_v1.md

`app/prompts/task_parser_v1.md` etc.

The actual instructions given to the AI

Instead of writing AI instructions inside Python code (which is hard to edit and version), every agent's prompt is stored as a readable markdown file. You can read, edit, and improve them without touching any Python.

- `v1.md`, `v2.md` naming means you can improve prompts without breaking the old version
- `task_parser_v1.md` tells the AI exactly how to extract task fields from speech
- `finance_agent_v1.md` tells the AI how to generate spending insights
- `debrief_agent_v1.md` tells the AI how to run a post-event debrief conversation
- Editing a prompt file is the fastest way to improve AI behaviour

`tests/test_scoring.py`

Automated tests for the pressure scoring logic

Tests check that the scoring logic works correctly. For example: a task due tomorrow should score higher than a task due next month. Run tests with: `pytest tests/`

- Tests run automatically and tell you if something broke
- The scoring service has no AI so it is easy and fast to test
- Green tests = confidence that pressure scores are correct before showing them to users

04. Libraries Explained

Every library in requirements.txt was chosen for a specific reason. Here is what each one does and why it is in Orbi.

Web Framework

Library	Version	What it does	More detail
fastapi	0.115.0	The web framework — the backbone of the entire backend	Creates all the HTTP endpoints, handles routing, validates requests automatically, and generates the interactive /docs page you can use to test the API in a browser. FastAPI is async by default which means it can handle many users at once efficiently.
uvicorn[standard]	0.30.0	The server that runs FastAPI	FastAPI creates the app but needs a server to actually listen for requests. Uvicorn is that server. The [standard] part adds extras for better performance and WebSocket support needed for real-time bubble updates later.

Database

Library	Version	What it does	More detail
supabase	2.5.0	The official Python client for Supabase	Lets your Python code talk to the Supabase database and use Supabase Auth to verify user tokens. Without this library, you would have to write raw HTTP requests to the Supabase API manually.
psycopg2-binary	2.9.9	Low-level PostgreSQL connection driver	The underlying tool that actually opens a connection to Postgres. Supabase uses Postgres behind the scenes so this is required. The -binary version includes everything compiled and ready — no extra setup needed.
pgvector	0.3.2	Enables vector/semantic search in Postgres	Adds support for storing and searching AI embeddings in the database. This is what powers 'find that car insurance thing' — it searches by meaning rather than exact keywords. Used in Phase 2 onwards.

Data Validation

Library	Version	What it does	More detail
pydantic	2.7.0	Data validation and shape enforcement	When the phone app sends a request, Pydantic checks that every field is the right type and shape before the code touches it. Wrong data type = automatic error returned. This prevents an entire category of bugs. All the models in app/models/ are Pydantic classes.

pydantic-settings	2.3.0	Reads environment variables into typed Python objects	Loads values from .env into Python settings classes with type checking. So instead of <code>os.environ.get('SUPABASE_URL')</code> everywhere, you get <code>settings.supabase_url</code> with auto-completion and validation.
python-dotenv	1.0.1	Loads .env file into environment	Reads your .env file when the app starts so all the secret keys are available to the app. Without this, you would need to set environment variables manually every time you start the server.

AI Providers

Library	Version	What it does	More detail
groq	0.9.0	Official Groq Python client — primary AI for free and pro users	Connects to Groq's servers to run Llama 3.1 8B. Groq is used instead of running a model locally because it is extremely fast and cheap — roughly 50x cheaper than Claude per token. Free and pro tier users go through this.
anthropic	0.34.0	Official Anthropic Python client — AI for premium users	Connects to Anthropic's API to use Claude Sonnet 4.6 and Opus 4.6. Used only for premium tier users who pay £9.99/month. More powerful and more expensive than Groq. The <code>ai_router.py</code> decides when to call this.

HTTP & Networking

Library	Version	What it does	More detail
httpx	0.27.0	Async HTTP client for making requests to external services	Used internally by the Supabase and Groq clients and for any direct HTTP calls the backend needs to make (e.g. webhooks, external integrations in later phases). Async-native so it doesn't block the server while waiting for responses.

Security & Auth

Library	Version	What it does	More detail
python-jose[cryptography]	3.3.0	Reads and verifies JWT tokens from Supabase Auth	When the phone app logs in via Supabase, it receives a JWT token. This library lets the backend verify that token is genuine and extract the <code>user_id</code> from it. Every protected endpoint uses this.
passlib[bcrypt]	1.7.4	Password hashing utility	If any custom auth flows are needed beyond Supabase Auth, this library hashes passwords securely using <code>bcrypt</code> . Hashing means passwords are never stored as plain text — even if the database is compromised.

python-multipart	0.0.9	Handles file uploads in HTTP requests	Required for FastAPI to process form data and file uploads — used when users upload bank statement CSV files for finance import in Phase 5.
-------------------------	-------	--	---

Testing

Library	Version	What it does	More detail
pytest	8.2.0	The testing framework — runs all your automated tests	You write test functions and pytest finds and runs them all with one command: <code>pytest tests/</code> . It reports which tests passed and which failed with clear error messages. Professional codebases run tests before every deployment.
pytest-asyncio	0.23.7	Lets pytest test async Python functions	Since all Orbi's code uses <code>async/await</code> , the tests also need to be async. This plugin makes pytest understand async test functions. Without it, async tests would silently pass without actually running.

05. How Data Flows — A Real Example

Let's trace exactly what happens when a user says 'Remind me to pay rent on Friday' from their phone.

1	Phone app	Records voice, Whisper transcribes it locally to text. Sends POST /api/v1/tasks with the text and the user's JWT token.
2	main.py	Receives the HTTP request and routes it to the tasks router.
3	routers/tasks.py	Checks the JWT token is valid using python-jose. Extracts user_id. Calls the task service. Returns the result.
4	agents/coordinator.py	Detects intent is 'create task'. Routes to task_parser agent.
5	agents/task_parser.py	Loads prompt from prompts/task_parser_v1.md. Calls ai_router with the text and user's tier.
6	services/ai_router.py	Checks user tier. Free/Pro → calls Groq Llama 8B. Premium → calls Claude Sonnet 4.6. Gets back structured JSON.
7	services/scoring.py	Calculates initial pressure_score based on Friday due date and importance. Returns a number like 6.4.
8	db/tasks.py	Inserts the new TaskBubble into Supabase Postgres with all fields including owner_id and visibility=private.
9	Supabase Realtime	Database change triggers a realtime event pushed to the phone.
10	Phone app	Receives the new bubble. Renders it in the universe at the correct size based on pressure_score. Rent bubble appears.

06. What's Next — Phase 2 Preview

Phase 1 is complete. The skeleton is built. Here is what Phase 2 adds on top of this foundation:

→	Groq API connection Wire up the actual Llama 3.1 8B model so the task_parser and finance_agent can start making real AI calls. Right now ai_router.py has the structure but the actual API keys need connecting.
→	Pressure scoring live Connect the scoring service to real tasks from the database. Every bubble gets a live pressure score that recalculates when tasks change.
→	Finance categorizer live The rule-based categorizer runs on every new FinanceEntry automatically. Log 'Tesco £45' and it instantly becomes category: groceries.
→	Embeddings + pgvector Generate vector embeddings for every task and finance entry. Store them in pgvector. This enables the semantic search — 'what was that car insurance thing'.
→	Finance insights The finance_agent runs daily and generates spending observations using Groq. 'You spent 40% more on dining this month' — pushed as a notification.
→	Claude integration Wire Claude Sonnet 4.6 into ai_router for premium tier users. Premium users get richer responses, deeper reasoning, better memory synthesis.

You are 14% done. Phase 2 starts now.
Tell Claude: "Phase 2 ready" to get the full Phase 2 instructions.